

Introducción a los Sistemas Operativos / Conceptos de Sistemas Operativos

Tema 2 – Anexo II

Cambio de Contexto: paso a paso



- ✓ Versión: Septiembre 2026
- ✓ Palabras Claves: Cambio de Contexto, Cambio de Modo, Interrupción de Clock, Scheduler, Dispatcher, PCB, Pila de Kernel, TLB

Los temas vistos en estas diapositivas han sido mayormente extraídos de: Stallings, W. *Operating Systems: Internals and Design Principles*; Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*.



Escenario

- ✓ Se analiza un **cambio de contexto** provocado por el **fin del quantum** del proceso en ejecución
 - ✓ Planificación apropiativa (por ejemplo, Round Robin)
- ✓ Requiere una coordinación estricta entre:
 - ✓ **Hardware**: circuitos de la CPU
 - ✓ **Software**: rutinas del Kernel (ISR, Scheduler y Dispatcher)
- ✓ El ciclo completo se organiza en **4 fases** y **14 pasos**



Visión general



Fase 1 – Interrupción

Transición realizada íntegramente por el hardware



Fase 1 – Interrupción de clock

✓ **Paso 1 – Interrupción de clock:** el temporizador llega a cero y activa la línea de interrupción de la CPU

✓ Es lo que le permite al Kernel recuperar el control de la CPU

◆ *Por ejemplo, en los procesadores x86/x64 se utiliza el APIC timer local (antiguamente el PIT); en ARM, el Generic Timer a través del GIC*

✓ **Paso 2 – Detección:** al finalizar la instrucción en curso, la CPU verifica si hay interrupciones pendientes y habilitadas

✓ La instrucción en ejecución nunca queda a medias

◆ *Por ejemplo, en los procesadores x86/x64 se consulta el flag IF del registro RFLAGS/EFLAGS*



Fase 1 – Cambio de modo y de pila

✓ **Paso 3 – Cambio de modo:** el procesador pasa de Modo Usuario a Modo Kernel

✓ Desde aquí se pueden ejecutar instrucciones privilegiadas

◆ *Por ejemplo, en los procesadores x86/x64 el nivel de privilegio actual (CPL) lo indican los 2 bits bajos del selector CS: pasa de ring 3 a ring 0. En ARM, los bits de modo del CPSR / PSTATE*

✓ **Paso 4 – Cambio de pila:** la CPU deja la pila de usuario y pasa a la pila de kernel del proceso actual

✓ Cada proceso tiene su propia pila de kernel

◆ *Por ejemplo, en los procesadores x86/x64 la dirección se toma del TSS (campo RSP0) y se carga en RSP; en ARM, cada nivel de excepción tiene su propio SP*



Fase 1 – Resguardo mínimo y vector

✓ **Paso 5 – Resguardo mínimo:** el hardware guarda automáticamente lo indispensable para poder retornar: PC, registro de estado y SP de usuario

✓ Además, se enmascaran las interrupciones

◆ Por ejemplo, en los procesadores x86/x64 se apilan SS, RSP, RFLAGS, CS y RIP en la pila de kernel y se limpia IF; en ARM, se guardan en los registros ELR y SPSR

✓ **Paso 6 – Vector de interrupciones:** con el número de la interrupción se indexa la tabla de vectores, se obtiene la dirección de la ISR y se carga en el PC

◆ Por ejemplo, en los procesadores x86/x64 se usa la IDT, cuya dirección está en el registro IDTR (en modo real, la IVT)



Fase 2 – ISR y planificación

El Kernel completa el resguardo de información y decide qué proceso sigue



Fase 2 – Resguardo del contexto y atención

✓ **Paso 7 – Resguardo del contexto restante:** el preámbulo de la ISR apila en la pila de kernel los registros de propósito general

✓ El hardware solo guardó lo indispensable

◆ *Por ejemplo, en los procesadores x86/x64: RAX, RBX, RCX, RDX, RSI, RDI, RBP y R8 a R15*

✓ **Paso 8 – Atención del clock:**

✓ Se notifica el fin de la interrupción al controlador

✓ Se actualiza la hora del sistema y el uso de CPU

✓ Se decrementa el quantum del proceso en ejecución

◆ *Por ejemplo, en los procesadores x86/x64 se envía el EOI (End Of Interrupt) al APIC local*



Fase 2 – Estado de la pila de kernel

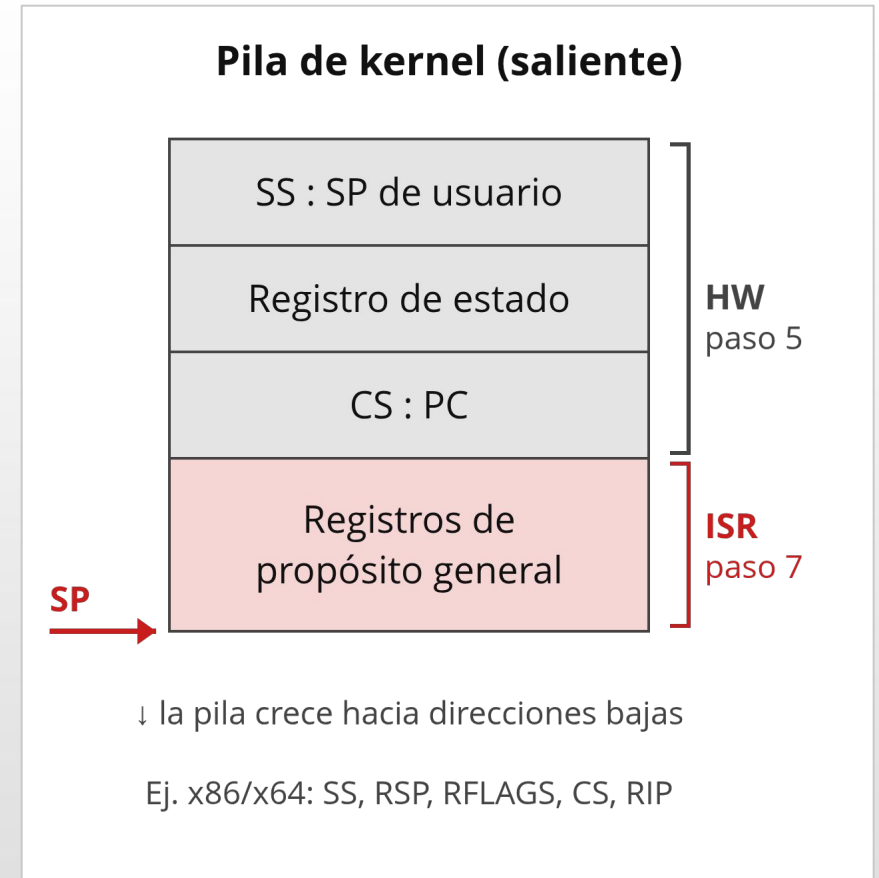
✓ Tras los pasos 5 y 7, la pila de kernel del saliente contiene su **contexto completo**

✓ El hardware apila lo mínimo (paso 5)

✓ La ISR apila el resto (paso 7)

✓ El **SP de kernel** queda apuntando a la cima de ese marco

✓ Ese valor es el que se guarda en el PCB (paso 10)



Fase 2 – Decisión del planificador

✓ **Paso 9 – Scheduler de corto plazo:** si el quantum se agotó, se lo invoca

- ✓ El proceso saliente pasa de **Running** a **Ready** y su PCB se encola en la cola de listos
- ✓ El algoritmo de planificación (RR, prioridades, etc.) elige el próximo proceso
- ✓ Si elige al mismo proceso, **no hay cambio de contexto**: se retorna directamente (Fase 4)
 - ◆ *Por ejemplo*, en Linux la ISR solo marca al proceso (TIF_NEED_RESCHED) y schedule() se invoca al salir de la interrupción, antes de volver a Modo Usuario



Fase 3 – Dispatcher

El cambio de contexto propiamente dicho



Fase 3 – Guardado del contexto

✓ El **Dispatcher** realiza los pasos 10 a 12

✓ **Paso 10 – Guardado del contexto saliente:** el SP de kernel, que apunta al marco salvado, se almacena en el PCB

✓ El PCB conserva así todo lo necesario para retomar al proceso

✓ Si corresponde, se guarda también el estado de la FPU y de los registros vectoriales

◆ *Por ejemplo, en los procesadores x86/x64 con las instrucciones FXSAVE / XSAVE*



Fase 3 – Espacio de direcciones

- ✓ **Paso 11 – Cambio de espacio de direcciones:** se carga el registro base de la tabla de páginas del proceso entrante
- ✓ Solo es necesario si el entrante tiene otro espacio de direcciones (no entre hilos del mismo proceso)
 - ✓ Efecto colateral: se invalidan las entradas de la TLB, lo que degrada el rendimiento inicial
 - ◆ Por ejemplo, en los procesadores x86/x64 se escribe el registro CR3 (PCID evita vaciar la TLB); en ARM, TTBR0 junto con el ASID



Fase 3 – Carga del proceso entrante

✓ **Paso 12 – Carga del contexto entrante:** el Dispatcher toma el PCB del proceso elegido, que pasa de **Ready** a **Running**

- ✓ Se actualiza el puntero a la pila de kernel para futuras interrupciones
 - ◆ *Por ejemplo, en los procesadores x86/x64 se escribe el campo RSP0 del TSS*
- ✓ Se restaura el SP desde el PCB: la CPU ya opera sobre la pila de kernel del nuevo proceso
- ✓ Se desapilan los registros de propósito general guardados cuando ese proceso fue suspendido



Fase 4 – Retorno a Modo Usuario

El proceso entrante retoma su ejecución



Fase 4 – Retorno a Modo Usuario

✓ **Paso 13 – Retorno de interrupción:** el Kernel ejecuta una instrucción atómica que desapila PC, registro de estado y SP de usuario

◆ *Por ejemplo, en los procesadores x86/x64 se utiliza IRET / IRETQ (o SYSRET si el proceso había sido suspendido en una system call); en ARM, ERET*

✓ **Paso 14 – Conmutación final:**

- ✓ El procesador vuelve a **Modo Usuario**
- ✓ El SP apunta a la pila de usuario del nuevo proceso
- ✓ El PC apunta a la instrucción donde el proceso fue suspendido: la reanudación es **transparente**



Cambio de modo vs. cambio de contexto

Cambio de modo

- ✓ Cambia el nivel de privilegio de la CPU
- ✓ Se mantiene el mismo proceso
- ✓ Ocurre en toda system call, interrupción o excepción
- ✓ Costo bajo

Cambio de contexto

- ✓ Cambia el proceso que usa la CPU
- ✓ Se ejecuta en Modo Kernel: siempre requiere un cambio de modo
- ✓ Guarda y restaura el contexto (PCB)
- ✓ Costo mayor: TLB y cachés "frías"



Ideas clave

- ✓ El hardware guarda **lo mínimo**; el Kernel guarda el resto
- ✓ El **Scheduler** decide qué proceso sigue; el **Dispatcher** efectúa el cambio
- ✓ Todo cambio de contexto implica cambios de modo, pero no a la inversa
- ✓ No todo fin de quantum produce un cambio de contexto
- ✓ El cambio de contexto es **overhead**: no realiza trabajo útil para los procesos

